Чек-лист интеграции

Английский оригинал: INTEGRATION_CHECKLIST.md.

Запускайте это в день интеграции (13 октября 2026) до слияния чего-либо в `main`. Каждый пункт должен быть отмечен; неотмеченный пункт блокирует слияние. Отмечайте пункт только после запуска команды или просмотра вывода, а не по памяти.

До слияния

- [ ] CI стадии захвата зелёный для текущей вершины. `gh run list --repo ForgeLabHub/ForgeMadrigalProtocolLab --branch capture --limit 3`
- [ ] CI стадии протокола зелёный для текущей вершины. `gh run list --repo ForgeLabHub/ForgeMadrigalProtocolLab --branch protocol --limit 3`
- [ ] Стадия протокола слила текущую `main` локально, и набор зелёный там, поэтому слияние — перемотка истории, а не сюрприз. `git checkout protocol && git merge main && pytest tests/ -q`
- [ ] Файлы, которые каждая ветка изменила в общих каталогах, не перекрываются, или перекрытие понято до начала. Общие места — `docs/`, `README.md`, `ARCHITECTURE.md`, `.gitignore` и `.github/workflows/`. `git diff --name-only main...capture > /tmp/a1.txt && git diff --name-only main...protocol > /tmp/a2.txt && comm -12 <(sort /tmp/a1.txt) <(sort /tmp/a2.txt)`
- [ ] `docs/CONTRACT.md` и обе схемы не изменены ни одной веткой. `git diff main -- docs/CONTRACT.md docs/schemas/`

Во время слияния

- [ ] Разрешите каждый конфликт вручную. Никогда не принимайте «ours» или «theirs» для целого файла и никогда не пушьте слияние, не прочитав разрешённый diff. `git --no-pager diff --check`
- [ ] Конфликт в любом из этих общих файлов эскалируется участнику, который решает; он не разрешается в одностороннем порядке: `docs/CONTRACT.md`, `.gitignore`, `README.md`, `ARCHITECTURE.md`, `.github/workflows/ci.yml`.
- [ ] Слитый `.github/workflows/ci.yml` сохраняет матрицу Python и ОС, работу качества кода и работу аудита из обеих веток.
- [ ] После разрешения `git --no-pager diff main..HEAD --stat` перечисляет только ожидаемые файлы.

После слияния

- [ ] Полный набор на `main`: `pytest tests/ -q` зелёный.
- [ ] CI на `main` зелёный. `gh run list --repo ForgeLabHub/ForgeMadrigalProtocolLab --branch main --limit 3`
- [ ] Конвейер проходит от начала до конца на корпусе: `python -m src.project.cli pipeline --pcap tests/corpus/corpus_capture_01.pcapng --project project.madrigal --report reports/REPORT.md`
- [ ] GUI открывает `reference_export/corpus_capture_01.normalized.json` (или соответствующий результат проекта) и перечисляет три сессии.

- [ ] Каждый файл в `tests/corpus/reference_export/` проходит проверку по `docs/schemas/capture.schema.json`. `python -m pytest tests/integration -q`
- [ ] Проект открывается после копирования в другой каталог. `cp -r project.madrigal /tmp/ && python -c "from src.project import Project; Project.open('/tmp/project.madrigal')"`
- [ ] Если ветка протокола добавляет документ эталонного исследования (например `docs/REFERENCE_INVESTIGATION.md`), его числа совпадают с `REPORT.md`: те же примеры контрпримеров, те же счётчики сообщений и то же покрытие корпуса. Несовпадение любого числа блокирует слияние, пока два документа не согласованы. Читайте числа из обоих документов; не носите их в этом чек-листе.

Эталонная выгрузка

- [ ] Три файла существуют в `tests/corpus/reference_export/`.
- [ ] Каждый проходит проверку по схеме захвата. `python -m pytest tests/integration -q`
- [ ] Выгрузки всё ещё совпадают с модулем. `bash tests/corpus/reference_export/regenerate.sh && git diff --stat`

Стадия протокола

- [ ] Применение `corpus_rule_v1.json` к каждому файлу в `reference_export/` завершается без ошибки и производит результат, проходящий проверку по `docs/schemas/result.schema.json`.
- [ ] На `corpus_capture_defects.normalized.json` сообщения, попадающие на `gap` и `ambiguity`, классифицируются как `incomplete` или `ambiguous`, никогда `matched`.
- [ ] Правило находит хотя бы один контрпример на `corpus_capture_02` (тот же протокол, другие значения).
- [ ] Изменение правила поднимает его версию и помечает предыдущий результат `outdated`.

Стадия GUI

- [ ] GUI открывает `reference_export/corpus_capture_01.normalized.json` и перечисляет три сессии.
- [ ] Hex-просмотр отрисовывает байты потока и подсвечивает диагностики на файле дефектов.
- [ ] Клик по байту показывает его индекс пакета, номер последовательности и временную метку из записи провенанса.
- [ ] Клик по контрпримеру прыгает к соответствующему байту в hex-просмотре.

Сквозной прогон

- [ ] Полный путь проходит на одном захвате: PCAP -> захват -> протокол -> GUI -> отчёт. `python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng --out out/cap01.json`
- [ ] Отчёт отрисовывается из исследования: `python -m pytest tests/report -q`, и HTML открывается без сети.
- [ ] Проект внутри `project.madrigal/` открывается после копирования в `/tmp`. `cp -r project.madrigal /tmp/ && python -c "from src.project import Project; Project.open('/tmp/project.madrigal')"`
- [ ] Проект экспортируется в zip и импортируется обратно с совпадающими контрольными суммами.

Документация

- [ ] `README.md` описывает текущие команды и указывает на верные документы.
- [ ] `ARCHITECTURE.md` соответствует тому, что построено, включая то, что не сделано.
- [ ] `docs/CAPTURE_API.md`, `docs/PROTOCOL_API.md` и `docs/INTEGRATION.md` актуальны.
- [ ] `REPORT.md` отражает фактически проведённое исследование.
- [ ] Ни одного `TODO` или `FIXME` не осталось, и ни один файл не называет инструмент, модель или ассистента. Регулярное выражение подписи живёт в страже отчёта, поэтому эта проверка не выписывает имена. `git grep -nE "TODO|FIXME"` и `pytest tests/integration/test_reference_report.py -k automation_traces`

Окружение

- [ ] Набор проходит в чистом контейнере на обоих концах поддерживаемого диапазона. `scripts/test_in_docker.sh ubuntu:22.04 3.12` и `scripts/test_in_docker.sh ubuntu:24.04 3.13`
- [ ] Свежий клон ставится и запускается без хоста разработки: `python3 -m venv .venv && source .venv/bin/activate && pip install -e ".[dev]" && pytest tests/ -q`
- [ ] Результаты в `docs/PORTABILITY.md` совпадают с только что выполненным прогоном.

Сдача

- [ ] Архив сдачи собирается и проверяется из чистого дерева: `python -m scripts.build_submission`
- [ ] Архив держит `SUBMISSION.md`, исходники, тесты и корпус, документацию и презентацию, и никакого кэша или виртуального окружения. `python -c "import zipfile,glob; print(zipfile.ZipFile(sorted(glob.glob('dist/submission_*.zip'))[-1]).namelist())"`
- [ ] Архив воспроизводит набор после распаковки; сборка сообщает «archive verified».

Полный прогон тестов

- [ ] `pytest tests/ -q` зелёный на `main` после слияний.
- [ ] Ни один захват, каталог проекта или `__pycache__` не закоммичен. `git status --short`